

SD File System Concepts

The two file types, group structure, overflow, on-disk layout, and how the hash algorithm works.

SD stores data in files, and each file is one of two types: a **directory file** or a **dynamic file**. The two have different on-disk layouts, different performance characteristics, and different rules for what can read them.

Directory files

A directory file is an **ordinary Windows folder** with one file per record. The record id is the file name.

On disk	C:\ProgramData\SD\user_accounts\<account>\<filename>\
Record id	the file name
Readable by	any Windows program (Notepad, Excel, etc.)
Record ids	matched case insensitively
<code>create.file</code> option	<code>no.case</code> for explicit case-insensitive ids

The `bp` file is a directory file. VOC, `batch.jobs`, `os.users`, and the dictionaries are all directory files.

SD writes directory file records with CR+LF line endings so that Windows programs can open them. Reading handles CR+LF, LF, and CR on its own.

Dynamic files

A dynamic file is stored in SD's own binary format. It has a hash-based group structure and resizes automatically as data grows or shrinks.

On disk	binary files in the account directory
Record id	up to <code>MAXIDLEN</code> , which is 63 by default

Readable by	SD only
Group size	set by <code>GRPSIZE</code> in <code>sd.conf</code> , 2 KB by default
Resizing	automatic, at a load of 80% to grow and 50% to shrink

GROUP STRUCTURE

A dynamic file is divided into **groups**. Each group is a bucket holding the records that hash to it. `analyse.file` reports the structure of a real file:

```
:analyse.file zzauditf
Account      : /cygdrive/c/ProgramData/SD/user_accounts/don
File name    : zzauditf
Path name    : /cygdrive/c/ProgramData/SD/user_accounts/don/ZZAUDITF
Type         : Dynamic, version 2
Group size   : 2 (2048 bytes)
Large record size : 1638
Minimum modulus : 1
Current modulus : 1
Load factors  : 80 (split), 50 (merge), 0 (current)
File size (bytes) : 6144 (4096 + 2048)
```

The **modulus** is the number of groups. A newly created file starts at 1.

IT GROWS ONE GROUP AT A TIME

This is the part most worth understanding, because the obvious guess is wrong.

The file does not double. When the load passes the split factor, SD adds **one** group and redistributes the records of **one** existing group into it. When the load falls below the merge factor, one group is merged away. The modulus therefore moves by one in either direction and is not a power of two.

Two consequences follow:

Growth is incremental	there is no pause while a large file is rebuilt, because no resize ever touches more than two groups
The cost is spread	a file that grows steadily splits steadily, rather than stalling at each doubling

`minimum modulus` is the floor: merging never takes the file below it. Set it with `configure.file` when you know a file will refill after being cleared.

OVERFLOW

When a group holds more data than fits in its primary buffer, further buffers are chained to it. That chain is **overflow**, and it has to be scanned on every read that lands in that group, so heavy overflow shows up as slow reads.

Overflow is normal in small amounts. It becomes a problem when a group holds records that will not fit however the file is sized — the *large record size* in the listing above is the threshold at which a record is stored separately instead.

`analyse.file` reports the distribution; `fstat` reports read and write statistics.

The dictionary file

Every data file has a companion dictionary, named `<FILENAME>.DIC` in the same account directory. `create.file` makes both parts and says so:

```
:create.file zzauditf
Created DICT part as ZZAUDITF.DIC
Created DATA part as ZZAUDITF
Added default '@ID' record to dictionary
```

`delete.file` removes both, and the VOC entry with them. The dictionary defines the fields for the query processor. See SD Dictionaries - Structure and SD Dictionaries - Conversions.

The sub-file structure

An account directory contains:

<code><FILENAME></code>	the data part (a folder for a directory file, binary for a dynamic one)
<code><FILENAME>.DIC</code>	the dictionary part
<code>voc</code>	the account's VOC
<code>bp</code>	BASIC source
<code>bp.out</code>	the compiled objects
<code>\$hold</code>	deferred print output
<code>\$svlists</code>	saved select lists, reached through the VOC name <code>\$savedlists</code>

File analysis

```
analyse.file customers
analyse.file customers statistics
fstat customers on
```

`analyse.file` the file's structure — type, group size, modulus, load factors and size, as shown above.
`statistics` adds the group distribution

`fstat` read and write counts. It is a **switch**: `fstat file on` starts collecting, `off` stops, and `fstat` alone reports. `fstat global` covers every file and `fstat reset` clears the counters

`fstat` collects nothing until it is turned on, so a report immediately after an `on` is empty rather than wrong.

Creating and configuring files

```
create.file customers
create.file orders directory
create.file dict customers
configure.file customers default
```

`create.file name` a dynamic file, which is the default. `directory` makes a directory file instead, and `dict` creates only the dictionary part

`delete.file name` removes the data part, the dictionary part and the VOC entry. `dict` or `data` removes only one part

`clear.file name` empties a file, keeping its structure. `dict` or `data` selects a part

`configure.file name` changes group size and load factors. `default` puts them back

There is no `static` file type. The two types are directory and dynamic.

`delete.file` refuses `voc` and `$acc`, and the refusal no longer depends on the case you type them in.
